

Here are the Step A commands. Run these on one box — this is a local loopback test, server and client both on the same machine.

1. Build keyd (go to wherever keyd.c is):



```
cc -O2 -Wall -o keyd keyd.c -lssl -lcrypto
```

If it fails with missing OpenSSL headers:



```
sudo dnf install openssl-devel      #  
RHEL/CentOS/Fedora  
# or: sudo apt install libssl-dev  #  
Debian/Ubuntu
```

then rerun the cc line.

2. Generate a test CA + two node certs:



```
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key
ca.key -days 3650 -subj "/CN=l2fwd-CA"
-out ca.crt
```

```
openssl genrsa -out nodeA.key 2048
openssl req -new -key nodeA.key -subj "/"
CN=nodeA" -out nodeA.csr
openssl x509 -req -in nodeA.csr -CA
ca.crt -CAkey ca.key -CAcreateserial
-days 3650 -out nodeA.crt
```

```
openssl genrsa -out nodeB.key 2048
openssl req -new -key nodeB.key -subj "/"
CN=nodeB" -out nodeB.csr
openssl x509 -req -in nodeB.csr -CA
ca.crt -CAkey ca.key -CAcreateserial
-days 3650 -out nodeB.crt
```

3. Make the socket directory:



```
sudo mkdir -p /run/l2fwd
```

4. Run the two keyd instances — open TWO terminals.

Terminal 1 (TLS server):



```
./keyd --listen --bind 127.0.0.1 --port  
4434 --rotate 10 \  
    --cert nodeA.crt --key nodeA.key --ca  
ca.crt --sock /run/l2fwd/keydA.sock
```

Terminal 2 (TLS client) — start this right after:



```
./keyd --connect 127.0.0.1 --port 4434 --  
rotate 10 \  

```

```
--cert nodeB.crt --key nodeB.key --ca  
ca.crt --sock /run/l2fwd/keydB.sock
```

What success looks like — both terminals print:



```
keyd: TLS up (TLSv1.3), peer verified  
keyd: installed epoch 0  
keyd: installed epoch 1    ← every 10  
seconds  
keyd: installed epoch 2  
...
```

(--rotate 10 makes it rotate every 10s so you see it quickly; the real system uses 120.)

Stop both with Ctrl+C when you've seen a few epochs tick.

Run it and tell me what the two terminals print —

especially whether you see peer verified and the epoch counter advancing.